

1 What is the model (EGMN)?

The main proposed model in this repository is **EGMN** (Exponential–Gaussian Mixture Network), implemented in `model/egmn.py`. EGMN models watch time as a **mixture distribution** designed to capture both short “quick-skip” behavior and longer watch-time behavior.

1.1 Distribution family

EGMN uses a mixture with:

- **One Exponential component** (to capture the spike near zero watch time).
- **Multiple Gaussian components** (to capture longer watch time). In the code, the Gaussian components are implemented as **Normals truncated at 0** (to avoid negative time).

If we denote the mixture weights by π , exponential rate by λ , and Gaussian parameters (μ_k, σ_k) , the model defines a mixture density over watch time $y \geq 0$ of the form:

$$p(y) = \pi_0 \text{Exp}(y; \lambda) + \sum_{k=1}^K \pi_k \text{TN}(y; \mu_k, \sigma_k, \text{lower} = 0),$$

where $\text{TN}(\cdot)$ is a Normal distribution truncated below at 0.

1.2 Neural architecture

- **Inputs:** a feature dictionary produced by the dataloader (sparse IDs, sequence features with masks, and continuous features such as **duration**).
- **Feature encoding:** embeddings for sparse/sequence features, and linear transforms for continuous features.
- **Shared trunk:** a multi-layer perceptron (MLP) over the concatenated feature representation.
- **Output heads:** predict mixture logits (converted to weights by softmax), the exponential rate λ (via softplus), and Gaussian parameters μ, σ (also via softplus for positivity constraints in the implementation).

1.3 Training objective (as implemented)

Training uses a combination of:

- **Negative log-likelihood (NLL)** of the observed watch time under the mixture.
- **Reconstruction regularizer:** an L1 loss between the observed label y and the model’s mixture *mean* prediction.
- **Mixture entropy term:** computed from the mixture weights π , weighted by a hyperparameter (`-alpha` in `run_egmn.py`).

In the training script, the total loss is:

$$\mathcal{L} = \mathcal{L}_{\text{NLL}} + \alpha \mathcal{L}_{\text{entropy}} + \beta \mathcal{L}_{\text{reg}},$$

with α and β controlled by command-line flags.

1.4 Prediction used for evaluation

For evaluation, the code uses a **point prediction equal to the mixture mean**. In the implementation, the predicted value is computed as a weighted sum of component means (including the exponential mean $1/\lambda$).